

Clasificación de sitios web de phishing mediante un árbol de decisión con scikit-learn

Jose Eduardo Perez Abundiz · A01666860

Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Querétaro

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Profesor: Dr. Benjamín Valdés

Resumen. Se implementó un árbol de decisión para clasificar phishing mediante 30 características. Se separaron 11,055 registros por perfiles idénticos y se compararon controles de complejidad. El modelo base obtuvo accuracy de 92.31 % y recall de phishing de 92.69 % en prueba, con 56 casos no detectados y 72 falsas alarmas. Los ajustes no superaron su F1 de validación.

Palabras clave / Keywords: phishing; clasificación binaria; árbol de decisión; scikit-learn; evaluación de modelos.

1. Introducción

El phishing consiste en suplantar entidades o servicios legítimos para engañar a los usuarios y obtener información sensible. Identificar estos sitios requiere considerar distintas señales, ya que una característica aislada no permite determinar de forma concluyente si una página es fraudulenta.

Este proyecto aborda la clasificación de sitios web mediante un árbol de decisión implementado con scikit-learn. Se utiliza el dataset Phishing Websites del repositorio UCI, compuesto por 11,055 registros con 30 características y una etiqueta que distingue sitios legítimos de sitios de phishing [1]. Las entradas representan propiedades previamente extraídas de las direcciones web, los dominios y el contenido o comportamiento de las páginas [2].

El trabajo comprende la revisión de los datos, su separación en entrenamiento, validación y prueba, la implementación del clasificador y el análisis de su desempeño. También se evalúa cómo cambia la clasificación al modificar la profundidad del árbol y el mínimo de registros por hoja.

El alcance se limita a clasificar los registros a partir de las características disponibles en el dataset. No se implementa la visita o inspección automática de páginas, ni se considera que los resultados garanticen la detección de cualquier amenaza actual.

1.1 Objetivo general

Implementar y evaluar un árbol de decisión con scikit-learn para clasificar registros de sitios web como legítimos o de phishing, analizando los errores de clasificación y el efecto de los parámetros de complejidad sobre su desempeño.

1.2 Justificación

La clasificación de sitios de phishing permite estudiar cómo distintas características de una página pueden contribuir a identificar registros fraudulentos. El aprendizaje supervisado ofrece una manera de combinar esas señales a partir de ejemplos etiquetados, en lugar de establecer que una sola característica determina la legitimidad del sitio.

La evaluación debe considerar tanto los casos de phishing clasificados como legítimos como los sitios legítimos

señalados incorrectamente. Los primeros representan amenazas no detectadas; los segundos, falsas alarmas. Por ello, se utilizan accuracy, precisión, recall y F1, acompañados de matrices de confusión. Se establece el F1 de phishing en validación como criterio principal para comparar configuraciones, manteniendo el recall como indicador relevante de la capacidad de detección.

1.3 Justificación del algoritmo

Se seleccionó un árbol de decisión porque permite representar la clasificación mediante una secuencia de condiciones sobre las variables de entrada. Esta estructura facilita examinar cómo se separan los registros y recorrer las reglas que conducen a una predicción [3].

El algoritmo permite controlar su complejidad mediante parámetros interpretables, como la profundidad máxima y el mínimo de registros por hoja. Estas propiedades son pertinentes para analizar la relación entre complejidad, ajuste al entrenamiento y desempeño en datos no utilizados para aprender.

La implementación con scikit-learn permite configurar el clasificador, entrenarlo y obtener predicciones mediante una biblioteca de aprendizaje automático. El trabajo se concentra en justificar la preparación de los datos, configurar el algoritmo y evaluar sus resultados. La elección responde a estos objetivos; no constituye una afirmación de superioridad frente a otras familias de modelos.

2. Descripción del conjunto de datos

Se utilizó el archivo Training Dataset.arff del conjunto Phishing Websites, disponible en UCI. Contiene 11,055 registros, 30 variables predictoras y una columna objetivo denominada Result [1].

Cada registro representa un perfil de características de un sitio web. El archivo no incluye las direcciones originales ni identificadores que permitan comprobar si dos registros corresponden al mismo sitio. Por tanto, dos filas con características idénticas no se interpretan automáticamente como una duplicación del sitio de origen.

Las variables describen propiedades de la dirección web, del dominio y del contenido o comportamiento de la página. Algunos ejemplos son el uso de una dirección IP en la URL,

la longitud de la dirección, las características asociadas a HTTPS y su certificado, la antigüedad del dominio y las propiedades de los enlaces [2].

Las entradas ya están codificadas mediante valores discretos. Algunas utilizan dos categorías y otras tres; su interpretación depende de la definición de cada variable. Estos códigos representan condiciones previamente extraídas, no mediciones directas de riesgo ni probabilidades de phishing.

2.1 Variable objetivo

Result utiliza -1 para phishing y 1 para sitios legítimos. Para facilitar la interpretación de las métricas, se creó `es_phishing`, asignando 1 a phishing y 0 a legítimo. La correspondencia es consistente con el uso de este dataset en la literatura [6].

Tabla 1. Correspondencia de etiquetas y distribución original.

Clase	Original	Utilizado	Registros
Legítimo	1	0	6,157
Phishing	-1	1	4,898
Total	-	-	11,055

La transformación modifica únicamente la codificación de la respuesta. Tanto Result como `es_phishing` se excluyeron de las variables predictoras, evitando proporcionar al modelo la información que debía aprender a estimar.

3. Preparación de los datos

3.1 Carga y revisión inicial

El archivo ARFF se cargó mediante SciPy y se convirtió en un DataFrame de Pandas. Las categorías se transformaron a valores enteros para trabajar con su codificación numérica. Se conservaron las 30 variables predictoras y se creó la etiqueta binaria `es_phishing`.

La revisión inicial no identificó valores faltantes representados mediante el marcador `?`. También se comprobó que la matriz de entrada contenía 11,055 filas y 30 columnas. La ausencia de valores faltantes no se interpretó como una garantía de que los registros estuvieran libres de inconsistencias.

3.2 Identificación de perfiles repetidos

Se compararon las combinaciones de las 30 características, sin incluir la variable objetivo. Se identificaron 5,785 perfiles distintos entre los 11,055 registros.

Tabla 2. Resultados de la revisión de perfiles.

Hallazgo	Cantidad
Registros originales	11,055
Perfiles distintos	5,785
Perfiles repetidos con una sola etiqueta	2,550
Perfiles con ambas etiquetas	64
Registros en perfiles con ambas etiquetas	357

Los 64 perfiles con ambas etiquetas presentan características idénticas, pero contienen registros clasificados como legítimos y como phishing. Esta situación no demuestra por sí sola un error de etiquetado: sitios diferentes pueden compartir la representación disponible. Sin embargo, evidencia una limitación, ya que un clasificador determinista que recibe únicamente esas características no puede distinguir perfectamente los registros contradictorios.

3.3 Conservación y agrupación

Se decidió conservar todos los registros, sin eliminar repeticiones ni sustituir etiquetas. El archivo no proporciona identificadores que permitan determinar si una repetición corresponde al mismo sitio, ni evidencia suficiente para establecer cuál etiqueta debería prevalecer en los perfiles contradictorios.

Para evitar que un perfil idéntico apareciera en entrenamiento y evaluación, se asignó un identificador de grupo a cada combinación de las 30 características. Todos los registros de un grupo se mantuvieron dentro de un mismo conjunto, incluso cuando sus etiquetas eran diferentes.

El identificador se utilizó exclusivamente para organizar la separación; no se incorporó como entrada del clasificador. Esta medida evita compartir perfiles exactos entre conjuntos, aunque no garantiza independencia entre sitios relacionados que presenten características diferentes.

Conservar las repeticiones implica que los perfiles más frecuentes mantienen mayor peso en el entrenamiento y en las métricas calculadas por registro. Este efecto se reconoce como parte del diseño de la evaluación.

4. Diseño de la evaluación

4.1 Separación por grupos

Se utilizó `GroupShuffleSplit` de scikit-learn para dividir los datos manteniendo completos los grupos de características idénticas. Primero se destinó aproximadamente el 70 % de los grupos a entrenamiento; los grupos restantes se dividieron entre validación y prueba. En ambas operaciones se fijó `random_state=42` para reproducir la separación [4].

Los porcentajes se aplican a grupos, no directamente a registros. Debido a que los grupos tienen tamaños diferentes, las proporciones finales de filas no coinciden exactamente con el reparto nominal de 70 %, 15 % y 15 %.

Tabla 3. Distribución por conjunto.

Conjunto	Registros	Total (%)
Entrenamiento	7,650	69.20
Validación	1,740	15.74
Prueba	1,665	15.06
Total	11,055	100.00

Tabla 4. Clases en cada conjunto.

Conjunto	Legítimos	Phishing
Entrenamiento	4,276	3,374
Validación	982	758
Prueba	899	766

Se verificó mediante comprobaciones programáticas que ningún grupo apareciera en más de un conjunto. La proporción de phishing fue de 44.10 % en entrenamiento, 43.56 % en validación y 46.01 % en prueba. La separación no fue estratificada por clase, pero conservó proporciones similares sin buscar otra partición en función del desempeño.

4.2 Función de cada conjunto

Entrenamiento se utilizó para aprender las reglas del árbol. Validación permitió comparar configuraciones y analizar errores, utilizando el F1 de phishing como criterio principal y revisando también precisión, recall y accuracy.

Tras los primeros experimentos de profundidad máxima y mínimo de registros por hoja, se conservó el árbol base y se evaluó en prueba sin volver a entrenarlo. La evaluación de prueba permitió medir el desempeño de esa configuración previamente seleccionada.

Posteriormente se realizó una exploración adicional de combinaciones de parámetros, utilizando únicamente entrenamiento y validación. Ninguna superó el F1 de validación del modelo base, por lo que no se modificó la configuración final. Esta exploración ocurrió después de conocer los resultados de prueba: no se presenta como una selección anterior a abrir ese conjunto ni como una nueva evaluación independiente.

4.3 Alcance de la evaluación

Las métricas describen el desempeño sobre una partición fija del dataset, con separación de perfiles idénticos. No equivalen a una evaluación temporal ni garantizan el mismo resultado frente a páginas actuales o provenientes de otras fuentes. Las comparaciones reiteradas sobre validación pueden favorecer decisiones particulares de ese conjunto; por ello, los resultados no demuestran superioridad general del algoritmo.

5. Fundamento e implementación

5.1 Funcionamiento del clasificador

Un árbol de decisión clasifica los registros mediante una secuencia de condiciones sobre sus características. Comienza en un nodo raíz, donde se evalúa una primera condición. Según el resultado, cada registro continúa por una de las ramas hasta llegar a una hoja, que proporciona la clasificación [3].

Durante el entrenamiento, el algoritmo selecciona variables y umbrales que permiten separar las clases. Las reglas no se escriben manualmente: se aprenden a partir de entradas y etiquetas conocidas. En esta implementación, la clase asignada en una hoja corresponde a la mayoritaria entre los registros de entrenamiento que llegaron a ella.

5.2 Impureza Gini

Se utilizó el criterio Gini para evaluar las divisiones. La impureza de un nodo se expresa como [3]:

$$G = 1 - \sum_{k=1}^K p_k^2$$

donde p_k representa la proporción de registros de la clase k dentro del nodo. En este problema existen dos clases: legítimo y phishing.

Si todos los registros pertenecen a una misma clase, la impureza es cero. Para dos clases, alcanza su máximo de 0.5 cuando ambas tienen la misma proporción. El algoritmo busca divisiones que reduzcan la impureza, considerando el tamaño de los nodos resultantes. No basta con obtener una rama pequeña y pura si la separación global aporta poca mejora.

5.3 Complejidad y generalización

La profundidad indica el número de divisiones desde la raíz hasta un nodo. Un árbol más profundo puede representar combinaciones más específicas, pero también puede ajustarse a particularidades del entrenamiento que no se repitan en otros registros [3].

La profundidad máxima y el mínimo de registros por hoja permiten controlar esta complejidad. Restringirlos puede reducir el sobreajuste; una restricción excesiva puede impedir aprender patrones útiles y producir subajuste. Su efecto debe comprobarse mediante validación, no suponerse a partir del tamaño del árbol.

5.4 Configuración e implementación

Se utilizó DecisionTreeClassifier de scikit-learn. El modelo base se configuró con Gini, sin límite explícito de profundidad, mínimo de un registro por hoja y semilla aleatoria 42. El método fit recibió únicamente las características y etiquetas de entrenamiento. Posteriormente, predict aplicó las reglas aprendidas para clasificar los registros de evaluación.

A diferencia de la regresión lineal implementada previamente, este árbol no aprende mediante descenso de gradiente ni requiere una activación sigmoide. Construye divisiones sucesivas: Gini orienta esas divisiones, mientras que accuracy, precisión, recall y F1 evalúan las predicciones.

No se aplicó normalización min-max: las entradas ya estaban codificadas y el árbol establece condiciones mediante umbrales [3]. Estos operan sobre códigos numéricos y deben interpretarse junto con el significado de cada variable, no como probabilidades de riesgo. Se mantuvo la representación ordinal original; no se evaluaron codificaciones alternativas.

La versión ejecutable modelo_framework.py integra carga, separación, experimentos y exportación de resultados. Guarda métricas, predicciones y figuras sin depender de un notebook. La ejecución verificada utilizó Python 3.13.15, scikit-learn 1.9.0, Pandas 3.0.5, SciPy 1.18.1 y Matplotlib 3.11.1. Las combinaciones adicionales se ejecutaron después, durante la exploración documentada, y no forman

parte del script inicial.

5.5 Interpretación de las reglas

La primera condición aprendida fue $SSLfinal_State \leq 0.5$. Como esta variable contiene -1, 0 y 1, separa los registros con valores -1 o 0 de aquellos con 1. El umbral no representa una probabilidad: opera sobre la codificación de la característica.

En la rama izquierda se evalúa $URL_of_Anchor \leq -0.5$. Los registros que cumplen ambas condiciones llegan a una hoja con 2,082 ejemplos de entrenamiento, todos etiquetados como phishing. La hoja asigna esa clase porque no contiene ejemplos legítimos.

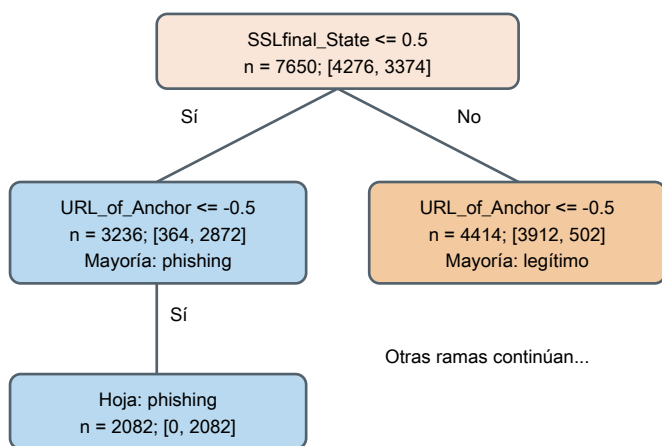


Figura 1. Extracto de las reglas aprendidas. Los pares indican [legítimos, phishing]. Se omiten otras ramas; n cuenta registros, no sitios únicos. Elaboración propia a partir del árbol base.

La regla combina características para clasificar. No demuestra que sea infalible fuera del entrenamiento ni que corresponda a 2,082 sitios distintos, pues se conservaron perfiles repetidos. La figura muestra solo una parte del árbol; limitar el dibujo no modifica el modelo.

6. Métricas de evaluación

Se consideró phishing como clase positiva y legítimo como negativa. Un verdadero positivo (VP) es phishing detectado; un verdadero negativo (VN), un legítimo reconocido. Un falso positivo (FP) es una falsa alarma sobre un legítimo, y un falso negativo (FN), phishing clasificado como legítimo.

Accuracy expresa la proporción total de clasificaciones correctas [5]:

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

Resume el desempeño global, pero no distingue falsas alarmas de phishing no detectado. La precisión de phishing indica qué proporción de los registros señalados pertenece realmente a esa clase [5]:

$$Precision = \frac{VP}{VP + FP}$$

Recall mide qué proporción de los casos reales de phishing fue detectada [5]:

$$Recall = \frac{VP}{VP + FN}$$

Este indicador importa porque los falsos negativos presentan registros fraudulentos como legítimos. Maximizar recall por sí solo sería insuficiente: clasificar todo como phishing detectaría todos los casos, pero produciría falsas alarmas sobre todos los legítimos.

F1 combina precisión y recall mediante su media armónica [5]:

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Se utilizó F1 de phishing como criterio principal de comparación en validación, revisando también las métricas individuales. F1 equilibra precisión y recall, pero no incorpora explícitamente un costo mayor para el phishing no detectado.

Las métricas se calcularon por registro con scikit-learn. En los reportes, support indica la cantidad de ejemplos reales de cada clase, no los aciertos. Los promedios macro asignan el mismo peso a ambas clases; los ponderados consideran su número de registros. Las matrices de confusión complementan los indicadores con cantidades concretas de errores [5].

7. Resultados y ajustes

7.1 Modelo base

El árbol base alcanzó profundidad 20 y 424 hojas. Obtuvo accuracy 0.9902 en entrenamiento y 0.9385 en validación; precisión de phishing 0.9934 y 0.9168; recall 0.9843 y 0.9446; F1 0.9888 y 0.9305, respectivamente. La tabla 7 reúne también prueba para facilitar la comparación.

La diferencia de accuracy fue aproximadamente 5.17 puntos porcentuales. El mayor desempeño en entrenamiento es compatible con sobreajuste y motivó evaluar configuraciones menos complejas. Esta diferencia no determina por sí sola su gravedad ni cuantifica formalmente la varianza.

7.2 Profundidad máxima

Se evaluaron profundidades 3, 5, 8, 12 y 20, manteniendo los demás parámetros y la partición.

Tabla 5. Ajuste de profundidad. Acc. = accuracy; F1 corresponde a phishing en validación.

Límite	Hojas	Acc. ent.	Acc. val.	F1 val.
3	7	0.9068	0.9167	0.8987
5	21	0.9265	0.9270	0.9130
8	93	0.9464	0.9305	0.9201
12	284	0.9778	0.9368	0.9277
20	424	0.9902	0.9293	0.9199

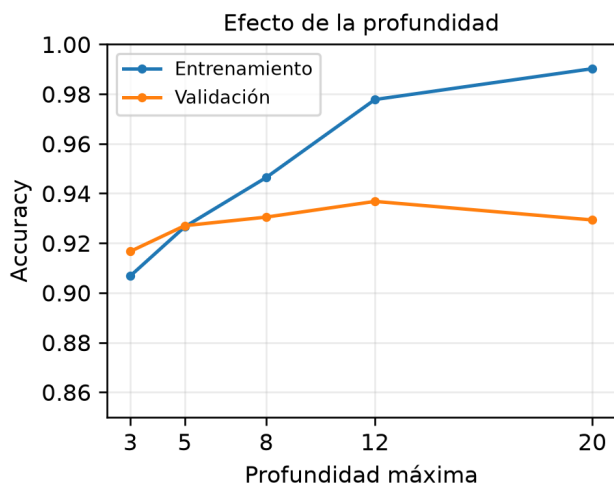


Figura 2. Efecto de la profundidad sobre accuracy. Cada punto es un modelo distinto, no una época. El base sin límite no aparece en esta curva. Fuente: elaboración propia.

Entrenamiento mejoró con la profundidad, mientras validación mejoró hasta 12 y disminuyó a 20. Añadir complejidad favoreció el ajuste conocido sin mejorar validación. La profundidad 3 obtuvo resultados inferiores en ambos conjuntos respecto a otras alternativas, compatible con capacidad insuficiente para algunos patrones.

El mejor F1 de esta serie fue 0.9277 (profundidad 12), por debajo de 0.9305 del base. Aunque el límite 20 y el base alcanzaron igual profundidad y número de hojas, produjeron resultados distintos. La diferencia se reprodujo al entrenarlos nuevamente con los mismos datos; no se determinó su causa interna. Se conservaron como configuraciones distintas sin sustituir sus resultados.

7.3 Mínimo por hoja

Se evaluaron mínimos de 1, 5, 10 y 20 registros por hoja, sin límite explícito de profundidad. Este parámetro restringe hojas sustentadas en pocos ejemplos.

Tabla 6. Ajuste del mínimo de registros por hoja.

Mínimo	Hojas	Acc. ent.	Acc. val.	F1 val.
1	424	0.9902	0.9385	0.9305
5	257	0.9663	0.9264	0.9148
10	175	0.9505	0.9356	0.9272
20	116	0.9413	0.9224	0.9114

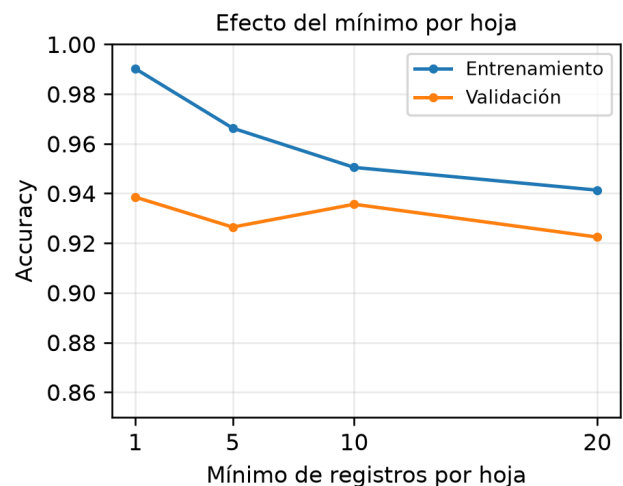


Figura 3. Accuracy al variar el mínimo por hoja. El valor 1 reproduce el base. Fuente: elaboración propia.

Aumentar el mínimo redujo las hojas, pero ninguna configuración superó el F1 base. Con mínimo 10 se obtuvo un árbol de 175 hojas y F1 0.9272: cercano, aunque inferior según el criterio fijado. La menor diferencia entre entrenamiento y validación ocurrió principalmente por la disminución del desempeño de entrenamiento. Simplificar y mejorar la predicción son resultados distintos.

7.4 Exploración adicional

Después de la evaluación inicial en prueba se exploraron nueve combinaciones de profundidad (8, 12, sin límite) y mínimo por hoja (2, 5, 10), exclusivamente con entrenamiento y validación. La mejor combinación adicional, sin límite y mínimo 2, obtuvo F1 0.9273 en validación. Profundidad 12 y mínimo 10 obtuvo 0.9272.

Ninguna alternativa superó el 0.9305 base, por lo que se conservó su configuración. Los experimentos documentan los controles de complejidad, pero no una mejora frente al modelo inicial ni una búsqueda exhaustiva de la configuración óptima.

8. Análisis de errores y prueba

8.1 Validación

El modelo base clasificó correctamente 1,633 de los 1,740 registros de validación. Detectó 716 de 758 casos de phishing (recall 94.46 %); dejó pasar 42. Asimismo, señaló 65 legítimos como phishing y reconoció correctamente 917 legítimos.

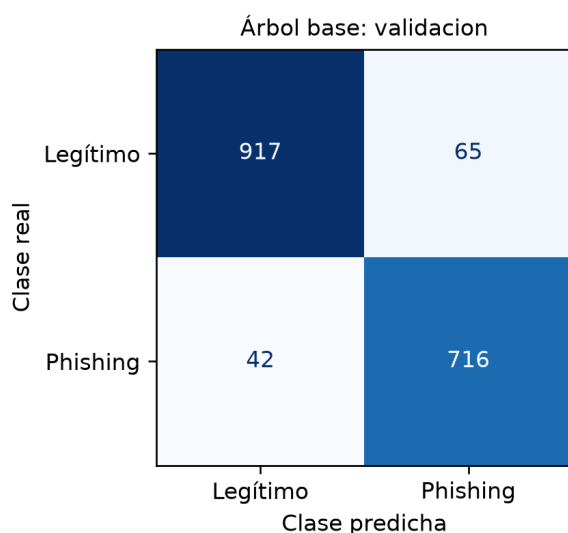


Figura 4. Matriz de confusión en validación. Filas: clase real; columnas: predicción. Fuente: elaboración propia.

Los errores tienen implicaciones diferentes. Un falso negativo puede producir falsa seguridad ante un registro fraudulento; un falso positivo genera una alerta incorrecta. El análisis considera detección y falsas alarmas.

8.2 Evaluación final

Seleccionada la configuración mediante validación, se evaluó sobre 1,665 registros de prueba sin volver a entrenar. La tabla siguiente compara los tres conjuntos; precisión, recall y F1 se refieren a phishing.

Tabla 7. Métricas del modelo seleccionado.

Métrica	Entren.	Valid.	Prueba
Accuracy	0.9902	0.9385	0.9231
Precisión	0.9934	0.9168	0.9079
Recall	0.9843	0.9446	0.9269
F1	0.9888	0.9305	0.9173

En prueba se clasificaron correctamente 1,537 registros y se cometieron 128 errores. Se detectaron 710 de 766 casos de phishing. Los 56 no detectados representan el 7.31 % de esa clase. De 899 legítimos, 72 se clasificaron como phishing y 827 correctamente.

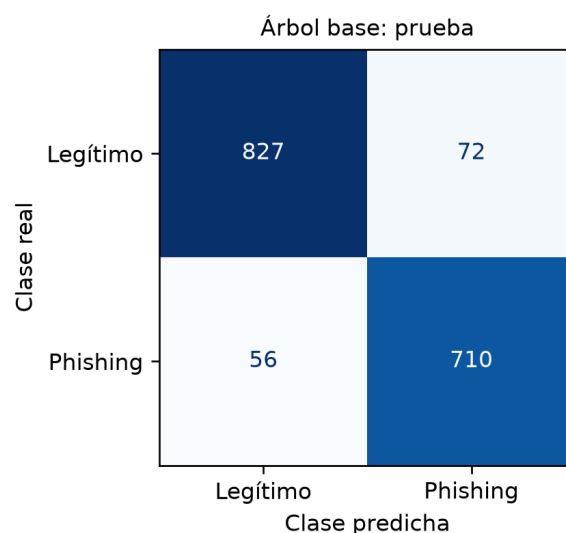


Figura 5. Matriz de confusión en prueba. Configuración fijada antes de esta evaluación. Fuente: elaboración propia.

La precisión de phishing fue 90.79 %: de los 782 registros señalados, 710 eran realmente de esa clase. Complementa el recall y muestra que una detección elevada no elimina falsas alarmas.

8.3 Interpretación

Accuracy de prueba fue aproximadamente 1.54 puntos porcentuales inferior al de validación. El modelo clasifica perfiles que no aparecieron de forma idéntica en entrenamiento, aunque conserva errores relevantes.

La cercanía entre validación y prueba no demuestra ausencia de sobreajuste. Ambos resultados permanecen por debajo del entrenamiento; una sola partición no permite establecer cuánto variarían las predicciones con otras muestras. La evaluación respalda un ejercicio supervisado sobre las características disponibles, no una garantía de seguridad ni una extrapolación automática a sitios actuales.

9. Sesgo, varianza y ajuste

El sesgo se relaciona con limitaciones para representar patrones; la varianza describe cuánto puede cambiar el modelo al entrenarse con muestras diferentes. Un modelo demasiado sencillo puede subajustar; uno excesivamente adaptado al entrenamiento puede sobreajustar. El diagnóstico aquí es cualitativo, basado en los resultados y controles de complejidad, no una descomposición formal ni una medición sobre múltiples particiones.

9.1 Sesgo

Se considera bajo de manera orientativa, dada la capacidad de ajuste al entrenamiento (accuracy 99.02 %). Este resultado no mide directamente el sesgo estadístico. Configuraciones restringidas como profundidad 3 mostraron resultados inferiores en ambos conjuntos respecto a otras alternativas, compatibles con representación demasiado sencilla.

Los perfiles contradictorios también limitan la clasificación perfecta: un árbol determinista no asigna respuestas distintas a entradas idénticas. No todo error puede atribuirse

exclusivamente a la complejidad elegida.

9.2 Varianza

Se valora como media de forma orientativa. La diferencia entre accuracy de entrenamiento (99.02 %), validación (93.85 %) y prueba (92.31 %) sugiere que parte del ajuste no se conserva en otros perfiles. La curva de profundidad refuerza la interpretación: de 12 a 20 mejora entrenamiento y empeora validación. Es una señal compatible con sensibilidad al entrenamiento, no una medición directa de cambios entre muestras.

9.3 Nivel de ajuste

El base presenta señales de sobreajuste, aunque conserva capacidad predictiva en evaluación. No se interpreta como subajuste generalizado, dado su elevado ajuste al entrenamiento. Reducir profundidad o aumentar mínimo por hoja no superó su F1 de validación; disminuir la brecha no justificó sustituirlo.

En síntesis: sesgo bajo, varianza media y señales de sobreajuste, como interpretación cualitativa y no niveles determinados por umbrales universales. Se conservó el base por validación, sin afirmar que estuviera libre de sobreajuste.

10. Limitaciones

Se usan características extraídas y codificadas, sin inspección automática ni identificación de sitios originales. El alcance es clasificar registros históricos, no implementar un servicio de seguridad para páginas actuales.

Se conservaron repeticiones y contradicciones. Separar perfiles exactos no garantiza separar sitios relacionados con características distintas; los perfiles frecuentes mantienen mayor peso en las métricas por registro.

Una sola partición no permite medir formalmente la estabilidad ni establecer intervalos de incertidumbre. El diagnóstico de sesgo y varianza es cualitativo. La exploración combinada ocurrió después de conocer prueba: aunque no la utilizó para ajustar ni cambió el modelo final, no se presenta como anterior a esa evaluación.

Los ajustes no mejoraron F1 del base. Esto no demuestra que ninguna configuración pueda mejorar, sino que las alternativas ensayadas no justificaron sustituirlo.

11. Conclusiones

Se implementó un árbol de decisión con scikit-learn capaz de clasificar registros legítimos y de phishing mediante 30 características. El programa integra preparación, separación por grupos, entrenamiento, comparación de parámetros y generación de métricas y figuras; puede ejecutarse como archivo Python sin notebook.

La revisión fue determinante para diseñar la evaluación. Los 11,055 registros contenían 5,785 perfiles distintos, incluidos 64 contradictorios. Mantener cada perfil en un conjunto evitó evaluar con combinaciones idénticas a las de entrenamiento, sin eliminar registros ni cambiar etiquetas sin evidencia.

El modelo obtuvo accuracy 92.31 % y F1 de phishing 0.9173 en prueba. Detectó 710 de 766 casos (recall 92.69 %),

mostrando capacidad para distinguir las clases bajo el diseño experimental. Sin embargo, los 56 casos no detectados y las 72 falsas alarmas delimitan ese logro. La matriz permite analizar estos errores sin reducir el resultado al porcentaje global ni interpretar predicciones como garantías de seguridad.

Los experimentos mostraron que reducir complejidad no asegura mejorar predicciones. Varias configuraciones redujeron hojas y brecha entre entrenamiento y validación, pero ninguna superó el F1 base. Se conservó este respetando el criterio fijado y se documentaron ajustes sin atribuirles una mejora inexistente.

El proyecto demostró el uso de una biblioteca de aprendizaje automático y permitió analizar representación, complejidad y desempeño. Alcanzó el objetivo de clasificar los registros disponibles, con errores y señales de sobreajuste que deben considerarse al interpretar sus resultados.

Referencias

- [1] Mohammad, R., y McCluskey, L. (2012). Phishing Websites [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C51W2X>
- [2] Mohammad, R. M., Thabtah, F., y McCluskey, L. (s. f.). Phishing Websites Features. Documento descriptivo adjunto al dataset de UCI.
- [3] Scikit-learn developers. (s. f.). Decision Trees. Documentación de scikit-learn. <https://scikit-learn.org/stable/modules/tree.html>
- [4] Scikit-learn developers. (s. f.). GroupShuffleSplit. Documentación de scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html
- [5] Scikit-learn developers. (s. f.). Metrics and scoring: quantifying the quality of predictions. Documentación de scikit-learn. https://scikit-learn.org/stable/modules/model_evaluation.html
- [6] Awasthi, A., y Goel, N. (2022). Phishing website prediction using base and ensemble classifier techniques with cross-validation. *Cybersecurity*, 5, artículo 22. <https://doi.org/10.1186/s42400-022-00126-9>